

Krishank Kureti

PES2UG23CS285

Sem 6 'E'

Lexer.l

```
%{
    #define YYSTYPE char*
    #include <unistd.h>
    #include "y.tab.h"
    #include <stdio.h>
    extern void yyerror(const char *); // declare the error handling function
}%

/* Regular definitions */
digit    [0-9]
letter   [a-zA-Z]
id       {letter}({letter}|{digit})*
digits   {digit}+
opFraction    (\.{digits})?
opExponent    ([Ee][+-]?{digits})?
number        {digits}{opFraction}{opExponent}
%option yylineno

%%
VV(.*) ; // ignore comments
[\t\n] ; // ignore whitespaces
"("          {return *yytext;}
")"          {return *yytext;}
"."          {return *yytext;}
","          {return *yytext;}
"*"          {return *yytext;}
"+"          {return *yytext;}
";"          {return *yytext;}
"_"          {return *yytext;}
"/"          {return *yytext;}
"="          {return *yytext;}
">"          {return *yytext;}
"<"          {return *yytext;}
```

```

{number}    {
                yylval = strdup(yytext); //stores the value of the number to be used later
for symbol table insertion
                return T_NUM;
            }
{id}        {
                yylval = strdup(yytext); //stores the identifier to be used
later for symbol table insertion
                return T_ID;
            }
.           {} // anything else => ignore
%%

```

Parser.y

```

%{
    #include "abstract_syntax_tree.c"
    #include <stdio.h>
    #include <stdlib.h>
    #include <string.h>
    void yyerror(char* s);
    // error handling function
    int yylex();
    // declare the function performing lexical analysis
    extern int yylineno;
    // track the line number
}%

%union
    // union to allow nodes to store return different datatypes
{
    char* text;
    expression_node* exp_node;
}

%token <text> T_ID T_NUM

%type <exp_node> E T F

/* specify start symbol */

```

```
%start START
```

```
%%
```

```
START : ASSGN      {  
                                printf("Valid syntax\n");  
                                YYACCEPT;  
                                // If program fits the grammar, syntax is valid  
                                }
```

```
/* Grammar for assignment */
```

```
ASSGN : T_ID '=' E  {  
                                expression_node* root = init_exp_node($1, NULL,  
$3);  
                                display_exp_tree(root);  
                                }  
;
```

```
/* Expression Grammar */
```

```
E : E '+' T      {  
                                $$ = init_exp_node("+", $1, $3);  
                                }  
    | E '-' T      {  
                                $$ = init_exp_node("-", $1, $3);  
                                }  
    | T      { $$ = $1; }  
;
```

```
T : T '*' F      {  
                                $$ = init_exp_node("*", $1, $3);  
                                }  
    | T '/' F {  
                                $$ = init_exp_node("/", $1, $3);  
                                }  
    | F {  
                                $$ = $1;  
                                }  
;
```

```
F : '(' E ')' { $$ = $2; }  
    | T_ID {  
                                $$ = init_exp_node($1, NULL, NULL);  
                                }
```

```

        | T_NUM      {
                                $$ = init_exp_node($1, NULL, NULL);
                                }
;

%%

/* error handling function */
void yyerror(char* s)
{
    printf("Error :%s at %d \n",s,yylineno);
}

/* main function - calls the yyparse() function which will in turn drive yylex() as well */
int main(int argc, char* argv[])
{
    yyparse();
    return 0;
}

```

Abstract_syntax_tree.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "abstract_syntax_tree.h"

expression_node* init_exp_node(char* val, expression_node* left, expression_node* right)
{
    expression_node* node = (expression_node*)malloc(sizeof(expression_node));
    node->val = strdup(val);
    node->left = left;
    node->right = right;
    return node;
}

void display_exp_tree(expression_node* exp_node)
{
    if (exp_node == NULL)
        return;
}

```

```

    printf("%s\n", exp_node->val);
    display_exp_tree(exp_node->left);
    display_exp_tree(exp_node->right);
}

```

Abstract_syntax_tree.h

```
typedef struct expression_node
```

```

{
    char* val;
    struct expression_node* left;
    struct expression_node* right;
} expression_node;

```

```

expression_node* init_exp_node(char* val, expression_node* left, expression_node* right);
void display_exp_tree(expression_node* exp_node);

```

Output

```

Last login: Thu Apr  2 12:26:28 on ttys000
krishankkureti@Krishanks-MacBook-Air PE5 % lex lexer.l
krishankkureti@Krishanks-MacBook-Air PE5 % bison -d parser.y
krishankkureti@Krishanks-MacBook-Air PE5 % ln -sf parser.tab.h y.tab.h
krishankkureti@Krishanks-MacBook-Air PE5 % ln -sf parser.tab.c y.tab.c
krishankkureti@Krishanks-MacBook-Air PE5 % gcc -g parser.tab.c lex.yy.c -ll -o a.out
ld: warning: object file (/Library/Developer/CommandLineTools/SDKs/MacOSX26.2.sdk/usr/lib/libl.a[arm64][3](libyywrap.o)) was built for newer 'macOS' version (26.2) than being
linked (26.0)
krishankkureti@Krishanks-MacBook-Air PE5 % ./a.out < test_input_1.c
a
+
-
/
10
5
*
2
7
3
Valid syntax
krishankkureti@Krishanks-MacBook-Air PE5 % ./a.out < test_input_2.c
b
-
+
/
c
6.7
12.45
*
a
1234.0
Valid syntax
krishankkureti@Krishanks-MacBook-Air PE5 % 

```